Reviewer Pack — Minimal Rank of Quotient Groups over Frege Proof Trees

Entry da7d9521a0fe · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 18:35:53 UTC

Minimal Rank of Quotient Groups over Frege Proof Trees

Entry ID: `da7d9521a0fe` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-26 18:35:53 UTC

1. Conjecture

Field A (mathematical branch): Group Theory (Quotient Groups) **Field B** (complexity object): Complexity Theory: Frege Proof Depth

Statement:

> ['For a given Frege proof tree T with depth D, there exists a quotient group G of finite order such that the minimal rank ρ(G) is upper-bounded by D.', 'Consequently, for all instances of Frege proof trees, ρ(G) ≤ D.', 'If an instance I of Frege proof trees does not satisfy ρ(G) ≤ D, then I cannot be represented as a tree of depth D.']

Rationale (proposer's reasoning):

> ['Quotient groups can capture the symmetries and abelianizations in complex structures. Applying this to Frege proof trees could reveal a connection between algebraic properties of the proofs and their computational complexity.', 'The depth of a proof tree is a fundamental measure of its complexity. A direct relationship between this and the minimal rank of quotient groups would provide new insights into the structure of proof systems.']

Taxonomy category: `GROUP_RANK` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `805ac4a7e78d019a`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the mean minimal rank ρ(G) across all 30 Frege proof trees is less than or equal to their respective depths D, and falsified if any tree has a minimal rank ρ(G) greater than its depth D.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.4s

5.1 Generated Python source

```python
import random
import math

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a*b) // gcd(a, b)

def matrix_multiply(A, B):
    m, n = len(A), len(B[0])
    p = len(B)
    C = [[0 for _ in range(n)] for _ in range(m)]
    for i in range(m):
        for j in range(n):
            for k in range(p):
                C[i][j] += A[i][k] * B[k][j]
    return C

def gaussian_elimination(A, b):
    m, n = len(A), len(A[0])
    augmented_matrix = [row + [b[i]] for i, row in enumerate(A)]
    for i in range(n):
        if i >= m:
            break
        max_row = i
        for j in range(i+1, m):
            if abs(augmented_matrix[j][i]) > abs(augmented_matrix[max_row][i]):
                max_row = j
        augmented_matrix[i], augmented_matrix[max_row] = augmented_matrix[max_row], augmented_matr
        pivot = augmented_matrix[i][i]
        for j in range(i, n+1):
            augmented_matrix[i][j] /= pivot
        for j in range(m):
            if j != i:
```

```
                factor = augmented_matrix[j][i]
                for k in range(i, n+1):
                    augmented_matrix[j][k] -= factor * augmented_matrix[i][k]
    return [row[-1] for row in augmented_matrix]

def minimal_rank(G):
    G = list(set(G))
    if not G:
        return 0
    A = [[gcd(G[i], G[j]) for j in range(len(G))] for i in range(len(G))]
    b = [1] * len(G)
    try:
        return len(gaussian_elimination(A, b))
    except Exception as e:
        print(f"Error in minimal_rank: {e}")
        return 0

def generate_frege_tree(depth):
    if depth == 0:
        return []
    else:
        left = generate_frege_tree(random.randint(0, depth-1))
        right = generate_frege_tree(random.randint(0, depth-1))
        return [left + right]

def run_trial(seed: int) -> dict:
    random.seed(seed)
    depths = range(1, 41)
    results = []
    for depth in depths:
        tree = generate_frege_tree(depth)
        literals = set()
        for node in tree:
            literals.update(node)
        G = list(literals)
        rho_G = minimal_rank(G)
        results.append({"depth": depth, "rho_G": rho_G})

    mean_value = sum(result["rho_G"] / result["depth"] for result in results) if any(result["depth
    support_fraction = sum(1 for result in results if result["rho_G"] <= result["depth"]) / len(re

    conjecture_holds = all(result["rho_G"] <= result["depth"] for result in results)
    counterexample = "" if conjecture_holds else f"Depth={results[0]['depth']}, rho(G)={results[0]

    return {
        "metric_name": "minimal_rank_over_depth",
        "metric_value": mean_value,
        "instances_tested": len(results),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 17 for i in range(30)]
    results = []
```

```
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={math.sqrt(sum((result['metric_value'] - m
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"Depth={results[0]['depth']}, rho(G)={results[0]
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_data")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fefda487.py", line 110, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fefda487.py", line 85, in run_trial
    literals.update(node)
TypeError: unhashable type: 'list'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed before producing any data, which means we cannot verify the mean minimal rank ρ(G) across all Frege proof trees against their respective depths D. | next: Re-run the test with proper error handling to ensure it completes and produces results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	22481
2	preregistration	ollama_remote	glm4:latest	0	0	5398
3	novelty	ollama_remote	glm4:latest	0	0	4714
4	novelty	ollama_remote	glm4:latest	0	0	4908
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18804
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10347
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10452
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	31660
9	judge	ollama_remote	glm4:latest	0	0	45292

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 154056 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/da7d9521a0fe.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/da7d9521a0fe.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/da7d9521a0fe.tar.gz` (if generated)